

CRL-Minimax: Algorithm Pseudocode

High-level pseudocode for the implemented continual-RL algorithms.

Abstract

This collection documents, as faithful high-level pseudocode, the three core algorithms of the CRL-Minimax continual reinforcement-learning system. A single shared-trunk, multi-head GLOBAL policy learns a sequence of tasks (Atari games) while retaining performance on all past tasks. The current method (Part A, config ppo.method = constrained) casts consolidation as a primal-dual min-max saddle: a throwaway LOCAL specialist supplies each task's value floor, and the global policy maximizes the weighted past-task objective subject to a one-sided value constraint. Experts are NOT stored; past-task quantities are re-estimated from fresh, replay-free rollouts. Each doc mirrors the code AS IMPLEMENTED, including its two faithful magnitude-only deviations, not an idealized version.

Source math: docs/Updated_Objective_for_CRL.pdf (Part A, eqs 1-48). This branch implements Part A only; Part B (stored expert models) has been stripped from the code.

Contents

03 Shared PPO Update Core. Source: `crl/ppo/trainer.py` (PPOTrainer.optimize_batches). The one reusable clipped-PPO optimizer shared by the local and global phases: per-stream actor-coeff times clipped surrogate, standard value MSE, entropy bonus, single global grad-norm clip. Actor coefficients are renormalized by their sum (prevents a large dual coefficient from starving the shared critic). ALL continual-learning weighting enters only through the actor coefficients.

04 Part A: Min-Max Consolidation, experts NOT stored (CURRENT method). Source: `crl/ppo_continual.py` (PPOAlternationTrainer), `crl/ppo/trainer.py` (Local/GlobalTrainer), `crl/duals/controllers.py` (ProjectedAscentDual). Three-checkpoint cycle piG-minus -> piL -> piG-plus. Stage 1 is plain PPO; stage $k \geq 2$ runs a local PPO phase producing the frozen floor V_k^L , then a global primal-dual phase maximizing the weighted past objective under the one-sided squared hinge $F_k = [V_k^L - V_k^G]_+^2 \leq \epsilon$, with current-task coeff $2\mu[V_k^L - V_k^G]_+$ and dual $\mu \leftarrow [\mu + \eta(F_k - \epsilon)]_+$. Replay-free; V are Monte-Carlo; only the actor is constrained. Verification: Part A derivation math-verifier SOUND; implementation code-verifier

PASS against eqs 26 / 38 / 40 / 47.

05 Windowed Off-Policy Expert Value-Agreement. Source: `crl/ppo/expert_eval.py`, `crl/envs/atari.py` (`make_snapshot_env`). A cheap off-policy DIAGNOSTIC of whether a policy matches its task's expert's windowed value from the expert's own states. BUILD (once per task): roll the expert greedily, `snapshot ale.cloneState()` each step, pick an adaptive horizon, keep consequential windows. EVALUATE (per checkpoint): restore $O(1)$, roll the policy greedily over the equal-length window, report the one-sided shortfall. Reported ALONGSIDE the greedy-100 anchor, never as a standalone forgetting number. Verification: code-verifier PASS against the design doc.

Reading guide

Read 03 first: it is the optimizer that 04's local and global phases both call. Then 04 for the continual-learning control flow, and 05 for the value-agreement diagnostic reported alongside greedy-100. Each PDF pairs its pseudocode with the equation numbers it realizes; the JSON sources under `specs/` are the regenerable inputs to `render_pseudocode.py`.